

Part 1: Onboarding ^

OpenClaw - Screening Course

9 sections • 100 min



Rubrics + Unit Tests Screening

16 sections • 132 min

OpenClaw RL - Universes

3 sections • 1 min



Almost Right: The Science of LLM Errors

5 sections • 62 min



OpenClaw RL Common Errors Screening

2 sections • 31 min

 Task Quality Module: Rubrics, Checks & Trajectories

6 sections • 46 min



OpenClaw Agentic Task Simulator



Unit Tests vs. Rubrics: When to Use Which

Unit Tests Verify Deterministic State

A unit test is appropriate **only when the correct answer is a single, deterministic outcome** derivable from the prompt + input data.

The test must pass for **any correct implementation**, not just the one your model happened to produce.

The Overfitting Trap

A unit test overfits to one model's output when it asserts specifics that are **one valid approach among many**. Whether an assertion is valid depends entirely on what the prompt and input data specify.

Part 1: Onboarding

OpenClaw - Screening Course
9 sections • 100 min

Rubrics + Unit Tests Screening
16 sections • 132 min

OpenClaw RL - Universes
3 sections • 1 min

Almost Right: The Science of LLM Errors
5 sections • 62 min

OpenClaw RL Common Errors Screening
2 sections • 31 min

Task Quality Module:
Rubrics, Checks & Trajectories
6 sections • 46 min

OpenClaw Agentic Task Simulator

Part 2: Tasking

A unit test overfits to one model's output when it asserts specifics that are **one valid approach among many**. Whether an assertion is valid depends entirely on what the prompt and input data specify.

Prompt** **says	Assertion	Verdict
"Save your analysis to a JSON file."	<code>assert filename == "analysis_v2.json"</code>	❌ Overfitting — prompt doesn't specify filename
"Save your analysis to a file called analysis_v2.json."	<code>assert filename == "analysis_v2.json"</code>	✅ Valid — prompt locks the filename
"Calculate an overall risk score for each vendor."	<code>assert score == 73.42</code>	❌ Overfitting — rounding, weighting method not fixed
"Calculate risk as (impact × 0.6) + (likelihood × 0.4). Vendor A has impact=80, likelihood=60."	<code>assert score == 72.0</code>	✅ Valid — formula + inputs = one answer
"Produce a structured comparison of the vendors."	<code>assert columns == ["risk", "impact", "score"]</code>	❌ Overfitting — column names not specified
"Output a CSV with columns: risk, impact, score."	<code>assert columns == ["risk", "impact", "score"]</code>	✅ Valid — prompt specifies exact column names
"Summarize your findings in a report."	<code>assert report.startswith("## Executive Summary")</code>	❌ Overfitting — heading style not specified
"Begin the report with a section titled '## Executive Summary'."	<code>assert report.startswith("## Executive Summary")</code>	✅ Valid — prompt specifies exact heading
"Send a follow-up email to the attendees."	<code>assert email["subject"] == "Follow-Up: Q3 Planning"</code>	❌ Overfitting — subject line not specified
"Send a follow-up email with subject 'Follow-Up: Q3 Planning'."	<code>assert email["subject"] == "Follow-Up: Q3 Planning"</code>	✅ Valid — prompt locks the subject

Part 1: Onboarding ^

OpenClaw - Screening Course

9 sections • 100 min

Rubrics + Unit Tests Screening

16 sections • 132 min

OpenClaw RL - Universes

3 sections • 1 min

Almost Right: The Science of LLM Errors

5 sections • 62 min

OpenClaw RL Common Errors Screening

2 sections • 31 min

Task Quality Module: Rubrics, Checks & Trajectories

6 sections • 46 min

OpenClaw Agentic Task Simulator

Part 2: Tasking

Openclaw Atlas

"Begin the report with a section titled '## Executive Summary'."	<code>assert report.startswith("## Executive Summary")</code>	Valid — prompt specifies exact heading
"Send a follow-up email to the attendees."	<code>assert email["subject"] == "Follow-Up: Q3 Planning"</code>	Overfitting — subject line not specified
"Send a follow-up email with subject 'Follow-Up: Q3 Planning'."	<code>assert email["subject"] == "Follow-Up: Q3 Planning"</code>	Valid — prompt locks the subject

The pattern: The assertion is only valid when the prompt (or input data) leaves **zero degrees of freedom** for that specific value. If the model had to *choose* that value rather than being *told* it, the test is overfitting.

The Decision Rule

Ask yourself: "Is there exactly one correct answer, fixed by the prompt and input data?"

Answer	Use
Yes — the prompt specifies an exact recipient, the CSV has exactly N rows, a deletion must remove a specific ID	Unit test. Assert the exact value.
No — the prompt says "structured output" (format flexible), "calculate a score" (naming flexible), "produce a summary" (content varies by interpretation)	Rubric. Evaluate the outcome against the prompt's intent.
Partially — some parts are deterministic, others aren't	Unit test the deterministic parts. Rubric the rest. Don't force a test onto the flexible parts.

Part 1: Onboarding ^

OpenClaw - Screening Course
9 sections • 100 min

Rubrics + Unit Tests Screening
16 sections • 132 min

OpenClaw RL - Universes
3 sections • 1 min

Almost Right: The Science of LLM Errors
5 sections • 62 min

OpenClaw RL Common Errors Screening
2 sections • 31 min

Task Quality Module: Rubrics, Checks & Trajectories
6 sections • 46 min

OpenClaw Agentic Task Simulator

Part 2: Tasking

Example: Same Task, Split Between Test and Rubric

Prompt: "Send a follow-up email to each of the 5 attendees listed in meeting_notes.csv. The email should thank them for attending, mention one key takeaway from the meeting, and ask if they have follow-up items."

What** to **check	Deterministic?	Approach
Were exactly 5 emails sent?	✔ Yes — 5 attendees in CSV	Unit test: <code>assert len(emails) == 5</code>
Were they sent to the right addresses?	✔ Yes — addresses are in the CSV	Unit test: <code>assert set(recipients) == {"alice@...", "bob@...", ...}</code>
Does each email thank them for attending?	✘ No — must contain thanks, but wording varies. There are many ways to reply as a thank you.	Rubric: The models includes a thank you note in each email
Does the email mention a key takeaway?	✘ No — which takeaway and how it's phrased is flexible	Rubric: "Each email references at least one specific discussion point from the meeting."
Does the email ask about follow-up items?	✘ No — must ask, but phrasing varies ("Do you have any follow-ups?"; "Let me know if you have any follow-up items."; "Please share any action items.")	BAD Unit test: <code>assert "?" in body</code> (weak) or Rubric (better) GOOD Rubric: "Email invites the recipient to share follow-up items."
Is the email well-written and professional?	✘ No — subjective	(don't test this at all)

Part 1: Onboarding ^

OpenClaw - Screening Course ✓

9 sections • 100 min

Rubrics + Unit Tests Screening

16 sections • 132 min

OpenClaw RL - Universes 🔒

3 sections • 1 min

Almost Right: The Science of LLM Errors 🔒

5 sections • 62 min

OpenClaw RL Common Errors Screening 🔒

2 sections • 31 min

🔍 Task Quality Module: Rubrics, Checks & Trajectories 🔒

6 sections • 46 min

OpenClaw Agentic Task Simulator 🔒

Part 2: Tasking 🔒 ▾

Unit Test Reminders

Common Mistakes

1. Testing format when the prompt specifies intent

Prompt: "Save your analysis as a JSON file."

- ✗ ``assert Path("analysis_output.json").exists()`` — filename not specified
- ✓ ``assert any(f.endswith(".json") for f in workspace_files)`` — tests the actual requirement

2. Testing exact numeric values when the data allows variation

Prompt: "Calculate average response time from the logs."

- ✗ ``assert avg == 247.83`` — depends on rounding, inclusion/exclusion of outliers
- ✓ ``assert 245 <= avg <= 250`` — if you can establish a reasonable tolerance from the data
- ✓ Use a **rubric** if even the tolerance isn't deterministic

3. Testing structure that reflects one model's style

Prompt: "Produce a ranked vendor comparison."

- ✗ ``assert "vendor_name" in columns and "composite_score" in columns`` — these exact key names aren't in the prompt
- ✓ ``assert len(vendors) >= 5`` — if the input data has 5+ vendors and the prompt says "rank the vendors"

Part 1: Onboarding ^

OpenClaw - Screening Course ✓
9 sections • 100 min

Rubrics + Unit Tests Screening
16 sections • 132 min

OpenClaw RL - Universes 🔒
3 sections • 1 min

Almost Right: The Science of LLM Errors 🔒
5 sections • 62 min

OpenClaw RL Common Errors Screening 🔒
2 sections • 31 min

🔍 Task Quality Module: Rubrics, Checks & Trajectories 🔒
6 sections • 46 min

OpenClaw Agentic Task Simulator 🔒

Part 2: Tasking 🔒 v

2. Testing exact numeric values when the data allows variation

Prompt: "Calculate average response time from the logs."

- ✗ ``assert avg == 247.83`` — depends on rounding, inclusion/exclusion of outliers
- ✓ ``assert 245 <= avg <= 250`` — if you can establish a reasonable tolerance from the data
- ✓ Use a **rubric** if even the tolerance isn't deterministic

3. Testing structure that reflects one model's style

Prompt: "Produce a ranked vendor comparison."

- ✗ ``assert "vendor_name" in columns and "composite_score" in columns`` — these exact key names aren't in the prompt
- ✓ ``assert len(vendors) >= 5`` — if the input data has 5+ vendors and the prompt says "rank the vendors"
- ✓ Use a **rubric** for "the comparison uses a multi-factor scoring methodology" — not unit-testable

4. Testing content that requires judgment

Prompt: "Write a risk assessment with mitigations."

- ✗ ``assert "supply chain" in report`` — which risks to highlight is model judgment
- ✓ ``assert report.count("###") >= 3`` — if the prompt says "include at least 3 sections"
- ✓ Use a **rubric** for "the report identifies at least 3 distinct risks with corresponding mitigations"

Part 1: Onboarding ^

OpenClaw - Screening Course 

9 sections • 100 min

Rubrics + Unit Tests Screening

16 sections • 132 min

OpenClaw RL - Universes 

3 sections • 1 min

Almost Right: The Science of LLM Errors 

5 sections • 62 min

OpenClaw RL Common Errors Screening 

2 sections • 31 min

 Task Quality Module: Rubrics, Checks & Trajectories 

6 sections • 46 min

OpenClaw Agentic Task Simulator 

Part 2: Tasking v

If In Doubt: Rubric

A bad unit test is worse than no unit test. It will:

- Fail correct implementations (false negative)
- Force future evaluators to delete it
- Create noise that obscures real model failures

When you aren't sure the outcome is deterministic — **use a rubric**. Reserve unit tests for assertions you'd bet money on: if a correct model ran 100 times with different phrasing, this assertion would pass every time.

Quick Self-Check Before Writing a Test

Before writing any assertion, pass it through these three filters:

1. **Is the expected value locked by the prompt or input data?** (not by what your model happened to output)
2. **Would a different correct implementation also pass this assertion?** (not just the one you saw)
3. **Can you point to the exact line in the prompt or row in the data that makes this the only valid answer?** (if not, it's a rubric)

All three must be yes. If any is no, delete the test or convert to a rubric.

Part 1: Onboarding ^

OpenClaw - Screening Course 

9 sections • 100 min

Rubrics + Unit Tests Screening

16 sections • 132 min

OpenClaw RL - Universes 

3 sections • 1 min

Almost Right: The Science of
LLM Errors 

5 sections • 62 min

OpenClaw RL Common Errors
Screening 

2 sections • 31 min

 Task Quality Module:
Rubrics, Checks & Trajectories 

6 sections • 46 min

OpenClaw Agentic Task
Simulator 

Part 2: Tasking v

Warning rubric generator

**Important: Rubric Generator**

The built-in rubric generator is designed to help you get started, but generated rubrics are often incomplete or inaccurate.

Before submitting, always:

- Review every generated criterion.
- Add any missing criteria.
- Ensure criteria are atomic, self-contained, and verifiable.
- Remove incorrect or redundant criteria.
- Verify all scores and justifications.

Remember: You are responsible for the final rubric quality, not the generator.

Outlier